

Parameter-Efficient Fine-Tuning for Mathematical Reasoning and Generalization

(Due date: **6 Oct 2026, 11:59pm**)

Introduction

In this assignment, you will adapt a small instruction-tuned large language model to solve grade-school mathematical word problems. You will first evaluate the original Qwen/Qwen2.5-1.5B-Instruct model, and then fine-tune it on the GSM8K training set using supervised fine-tuning (SFT) with 4-bit Quantized Low-Rank Adaptation (QLoRA) [2, 3]. The assignment focuses on two questions:

- Does SFT improve only in-domain GSM8K performance, or does it also improve generalization to related but different mathematical reasoning benchmarks?
- Does supervising intermediate reasoning steps provide benefits beyond supervising the final answer alone?

You will compare two SFT target formats—**answer only** and **reasoning steps plus answer**—and evaluate all model variants using deterministic, automatically computed metrics. The quality of the generated reasoning text will not be judged manually or by another language model.

Learning Objectives

After completing the assignment, you should be able to:

- construct instruction-tuning examples using a model’s chat template;
- perform memory-efficient SFT with 4-bit quantization and LoRA adapters;
- implement reproducible generation and numerical-answer evaluation;
- distinguish in-domain improvement from cross-dataset generalization; and
- conduct controlled experiments and communicate empirical findings clearly.

Skeleton Code and Environment Setup

A starter Jupyter Notebook will provide the main data-loading, model-loading, training, and evaluation structure. You may reuse or modify the provided code, but your final notebook must run in order and clearly record the settings and results of all required experiments.

The assignment is designed for a single NVIDIA T4 GPU available through free tiers such as Google Colab or Kaggle. Use the following stack unless the starter notebook specifies otherwise:

- PyTorch, Hugging Face transformers and datasets;
- peft for LoRA adapters; and
- bitsandbytes for 4-bit quantization.

Resource advice. Debug preprocessing, answer extraction, and evaluation on a small subset before launching full runs. Save adapter checkpoints and result files regularly because free GPU sessions may disconnect. Do not submit a copy of the full base model.

Base Language Model

Use Qwen/Qwen2.5-1.5B-Instruct, an approximately 1.5-billion-parameter instruction-tuned causal language model from the Qwen2.5 family [1]. The same model revision must be used for all required experiments. Do not replace it with a larger, math-specialized, or differently instruction-tuned model for the required comparison.

Data

GSM8K. GSM8K contains linguistically diverse grade-school mathematics word problems with worked solutions [4]. Use only its official training split for SFT. Create a fixed training/validation split from the official training data using the split and random seed provided in the starter notebook. The official test split is reserved exclusively for final evaluation.

SVAMP. SVAMP is a 1,000-example challenge set of arithmetic word problems designed to expose reliance on shallow patterns [5]. Treat the complete set as an external evaluation benchmark; it must not be used for training or model selection. Use the exact dataset versions and course-provided preprocessing code specified in the starter notebook. Do not silently replace a dataset with another Hugging Face conversion, because fields and answer formats may differ.

Required Output Format

All evaluated models must return an optional reasoning portion followed by exactly one final-answer line in the following format:

```
[optional reasoning steps]
#### <final numerical answer>
```

For example:

```
There are 12 + 8 = 20 items in total.
#### 20
```

The final line is mandatory. The answer-only SFT target should contain only the final line, whereas the reasoning-plus-answer target should contain the worked solution followed by the same final line. This common marker allows one evaluator to score every model variant.

Required Experiments

You must complete the following three model conditions and evaluate each condition on both benchmarks. This gives **six required evaluation runs**.

Model condition	Training	Assistant target during SFT / generation
Original model	None	Prompt the model to solve the problem and end with ##### <answer>.
SFT–Answer	4-bit QLoRA on GSM8K	##### <answer> only.
SFT–Reasoning	4-bit QLoRA on the same GSM8K examples	Worked solution followed by ##### <answer>.

Task 1: Data Processing and Evaluation Pipeline

1. Prepare the fixed GSM8K training/validation split and construct both SFT target formats.
2. Apply the Qwen chat template consistently. The user instruction should state the required final-answer format. The assistant response is the supervised target.
3. Implement a numerical answer extractor that reads the final ##### line. It should, at minimum, handle optional signs, commas, decimal values, and simple fractions. A malformed or missing answer must be recorded as an extraction failure rather than manually corrected.

Task 2: Original-Model Baseline

Evaluate the unmodified Qwen/Qwen2.5-1.5B-Instruct model on the GSM8K test set and the complete SVAMP set. Use a zero-shot prompt: benchmark questions or gold solutions may not be inserted as demonstrations.

For all final evaluations, use greedy deterministic decoding with `do_sample=False` and the generation limit specified in the starter notebook. Use the same prompt template and decoding settings for all three model conditions unless a difference is an explicit, separately reported optional experiment.

Task 3: 4-bit QLoRA Supervised Fine-Tuning

Train two LoRA adapters on the same GSM8K training examples:

1. **SFT–Answer:** supervise only the final ##### <answer> line.
2. **SFT–Reasoning:** supervise the provided worked solution and final-answer line.

Use 4-bit NF4 quantization, double quantization, and FP16 computation on a T4 GPU, unless the starter notebook provides an equivalent compatible configuration.

For each run, record at least the following:

- the number of training samples, the number of validation examples, random seed, and number of epochs;
- maximum sequence length, learning rate, effective batch size, optimizer, and scheduler;
- LoRA rank, alpha, dropout, and target modules;
- number and percentage of trainable parameters;
- training/validation loss, wall-clock training time, and peak GPU memory usage; and
- the final adapter checkpoint or sufficient configuration and logs to reproduce it.

Use the following suggested settings of hyperparameters for the required experiments:

- number of epochs: 2
- maximum sequence length: 512
- learning rate: 1e-5

For the required experiments, use the hyperparameter settings specified in the starter notebook, including the required 2 training epochs. Do not select settings based on GSM8K test or SVAMP results. Exploration of alternative hyperparameter settings is permitted as an optional extension after the required experiments have been completed.

Task 4: Evaluation and Comparative Analysis

Evaluate both fine-tuned adapters using exactly the same two benchmarks and final evaluation protocol as the original model. Your report must include:

1. a main table containing all model–benchmark combinations;
2. final-answer accuracy and answer-extraction success rate for every combination;
3. the cross-dataset generalization gap defined below;

Metrics

Let N_D be the number of examples in evaluation dataset D , e_i indicate that a valid numerical answer was extracted, and c_i indicate that the normalized extracted answer equals the normalized gold answer.

Final-answer accuracy

$$\text{Acc}(D) = \frac{1}{N_D} \sum_{i=1}^{N_D} \mathbb{1}[c_i]. \quad (1)$$

An extraction failure is counted as an incorrect answer. Numerical normalization must be deterministic and documented. Do not use an LLM judge.

Answer-extraction success rate (format compliance)

$$\text{Extract}(D) = \frac{1}{N_D} \sum_{i=1}^{N_D} \mathbb{1}[e_i]. \quad (2)$$

A successful output contains a parseable numerical answer after the required marker.

Conditional accuracy

Report conditional accuracy among successfully extracted answers as well, in addition to the final-answer accuracy. Conditional accuracy measures accuracy only among examples whose answers were successfully extracted. If no answers are extracted, it should return 0.0.

Cross-dataset generalization gap

$$\text{Gap}_{\text{SVAMP}} = \text{Acc}(\text{GSM8K}) - \text{Acc}(\text{SVAMP}). \quad (3)$$

A smaller positive gap indicates better retention of performance on the external benchmark. A negative gap is possible and must not be clipped. Also report the absolute accuracy change of each fine-tuned model relative to the original model on both benchmarks.

Experimental Controls and Data Integrity

The following requirements are part of the grading:

- For the required experiments, train only on the permitted GSM8K training examples. External math datasets are only allowed in the optional extensions.
- Use the fixed training/validation split and required hyperparameter settings for the main experiments. Do not use either test benchmark for model selection.

- Keep prompts, decoding settings, answer normalization, and evaluation code fixed across the required model comparison.
- Do not manually edit predictions or extraction results. Fixing a general bug and rerunning all affected models is allowed and must be documented.

Questions to Address in the Report

Your report should use evidence from your experiments to answer the following research question:

Does supervised fine-tuning on GSM8K improve cross-dataset mathematical generalization, or does it primarily improve in-domain performance and output formatting? Does supervision of intermediate reasoning steps help compared with answer-only supervision?

At minimum, discuss:

- whether in-domain improvements on GSM8K transfer to SVAMP;
- whether changes in accuracy can be separated from changes in format compliance (in particular, after finetuning, did the model become better at solving the mathematics, or did it mainly become better at following the required ##### <answer> output format?); and
- how answer-only and reasoning-plus-answer SFT differ in accuracy.

Optional Extensions

After completing all required experiments, you may investigate one or more controlled extensions. Possible directions include:

- **Error Analysis:** provide 2-4 representative examples of errors. This can include case studies or any bottlenecks for improvement that you identified, based on the saved prediction files.
- **Hyperparameter exploration:** investigate different learning rates, LoRA ranks, LoRA alpha values, batch sizes, maximum sequence lengths, or numbers of training epochs.
- **Additional training data:** incorporate another appropriate mathematical reasoning dataset and examine whether it improves in-domain or cross-dataset performance.
- **Data cleaning or preprocessing:** design and implement your own data filtering, cleaning, normalization, quality-control procedure, or modifications to the prompt.

Optional results must be clearly separated from the required experiments and cannot compensate for missing required runs.

Submission

The followings summarize the files that you need to submit. Please read the “Assignment2_Submission_Instructions.pdf” for the details about the format of each file and the structure of the submission directory.

Notebook, code, and result files (30%). Submit one ZIP archive named `Assignment2_<StudentID>.zip` containing a fully executed Jupyter Notebook with your implementation and experiments, together with all result files generated by the provided evaluation functions.

Submit the complete `submission/` directory generated by the Notebook. Do not rename, delete, or manually modify these files. This directory must include:

- `scores.csv`: the summary of the six required experiments;

- six *_metrics.json files: aggregate metrics for each model and benchmark;
- six *_predictions.jsonl files: per-example outputs, extracted answers, and correctness flags.

Also submit the following three files in a separate logs/ directory, alongside submission/:

- sft_answer_train_history.json;
- sft_reasoning_train_history.json;
- split_manifest.json.

Submit the completed Assignment2_Student_ready.ipynb. If you use Google Colab, download the Notebook only after saving its outputs. Download all result files from the temporary Colab workspace as well.

Report (70%). Submit the final report as report.pdf, using the ACL proceedings LaTeX format. Do not use a Word template. The main text must be **2–5 pages**, excluding references, appendices, and the “Use of Generative AI” declaration. Optional extensions may be included in an appendix.

The report should include:

1. a concise description of the experimental setup, including the model, data, principal QLoRA hyperparameters, prompting format, and evaluation procedure;
2. one main table reporting final-answer accuracy, answer-extraction success rate, and the cross-dataset generalization gap for all required model–benchmark combinations;
3. an evidence-based discussion of whether GSM8K fine-tuning improves cross-dataset generalization and whether reasoning-plus-answer supervision is more effective than answer-only supervision;

Optional experiments, tables, and error examples may be placed in an appendix.

The grading of the report will emphasize experimental setup and clarity, completeness and correctness of results, comparative analysis and interpretation, and presentation quality.

Use of Generative AI

Use of generative AI is **permitted within the course policy**. You may use such tools for explanations, brainstorming, debugging suggestions, and language editing. You remain responsible for understanding, testing, and verifying every submitted component.

You may not use another language model to generate the required benchmark predictions, reconstruct gold answers, manually repair the assigned model’s outputs, or write an analysis that you have not verified against your own experimental results. Discussion with classmates remains subject to the course’s academic-integrity rules.

Add a section titled “Use of Generative AI” at the end of your report. Identify the tool(s), explain the purposes for which they were used, and state how you checked the generated material. A suitable declaration is:

I acknowledge the use of [tool name and hyperlink] to assist with [specific purposes]. I reviewed, tested, and revised the generated material and take responsibility for the submitted work. No external model was used to generate or alter the required benchmark predictions.

University guidance is available at <https://ctle.um.edu.mo/resource/learning-with-generative-ai-at-um-module/>.

References

- [1] Qwen Team et al. 2024. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115*.

- [2] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*.
- [3] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. QLoRA: Efficient Finetuning of Quantized LLMs. In *Advances in Neural Information Processing Systems*.
- [4] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman 2021. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168*.
- [5] Arkil Patel, Satwik Bhattamishra, and Navin Goyal. 2021. Are NLP Models Really Able to Solve Simple Math Word Problems? In *Proceedings of NAACL-HLT*.
- [6] Quintong Li, Leyang Cui, Xueliang Zhao, Lingpeng Kong, and Wei Bi. 2024. GSM-Plus: A Comprehensive Benchmark for Evaluating the Robustness of LLMs as Mathematical Problem Solvers. *arXiv preprint arXiv:2402.19255*.